Assignment 5 – Sentiment Analysis of Financial Tweets

Author: Albert Borchardt (albert.borchardt@student.uni-tuebingen.de, 6871987)

Introduction

The GameStop saga, Elon Musk's tweets, market reactions to statements from politicians, etc. are striking examples of social media's growing role in financial markets and how viral online sentiment can instantly move prices. In this assignment, I decided to tackle the task of classifying financial tweets as **Bearish**, **Bullish**, or **Neutral**. I compare a feature-based logistic regression model with a neural model leveraging FinBERT, aiming to understand their strengths and limitations for extracting sentiment from short, finance-related tweets.

Data Exploration

I used the zeroshot/twitter-financial-news-sentiment dataset from Hugging Face. It consists of 9,543 training and 2,388 test entries, each with a short text and a sentiment label (0: Bearish, 1: Bullish, 2: Neutral) and the goal was to classify each test entry correctly into one of the three sentiment classes. Regarding the dataset it could be seen that it is very imbalanced—Neutral is the majority class, followed by Bullish and Bearish as seen in Figure 1 (this strong class imbalance later posed challenges for model training and evaluation, particularly in accurately identifying the less frequent Bearish and Bullish sentiments).

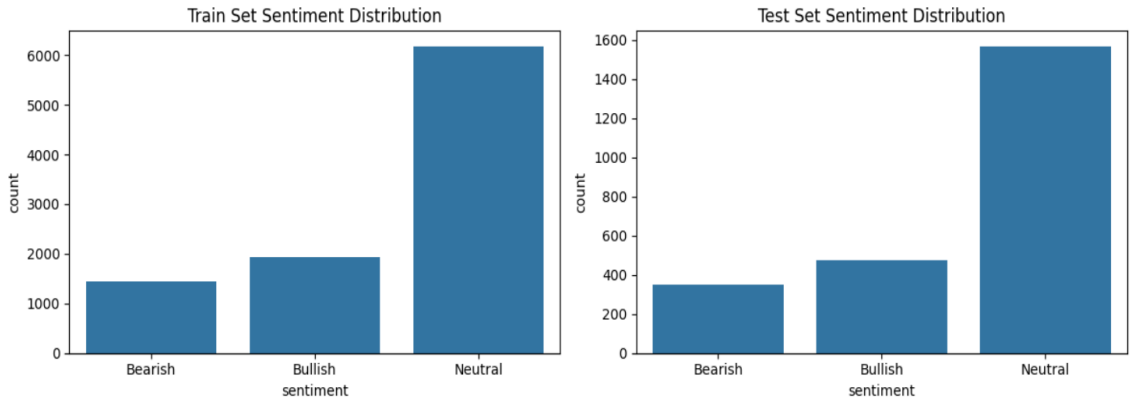


Figure 1: Class distribution in training and test data

But first I began by examining raw entries to understand the data's content and structure and as reflected in Figure 2 most entries contain ticker symbols, hashtags, and financial terminology (which was important information for the later preprocessing steps) and in general, the texts are short (on average 12 words as indicated in Figure 3), with no HTML markup or major typos.

```
First few entries:
                                                text  label
0  $BYND - JPMorgan reels in expectations on Beyo...      0
1  $CCL $RCL - Nomura points to bookings weakness...      0
2  $CX - Cemex cut at Credit Suisse, J.P. Morgan ...      0
3  $ESS: BTIG Research cuts to Neutral https://t....      0
4  $FNKO - Funko slides after Piper Jaffray PT cu...      0
```

Figure 2: Example tweets and sentiment labels from the dataset

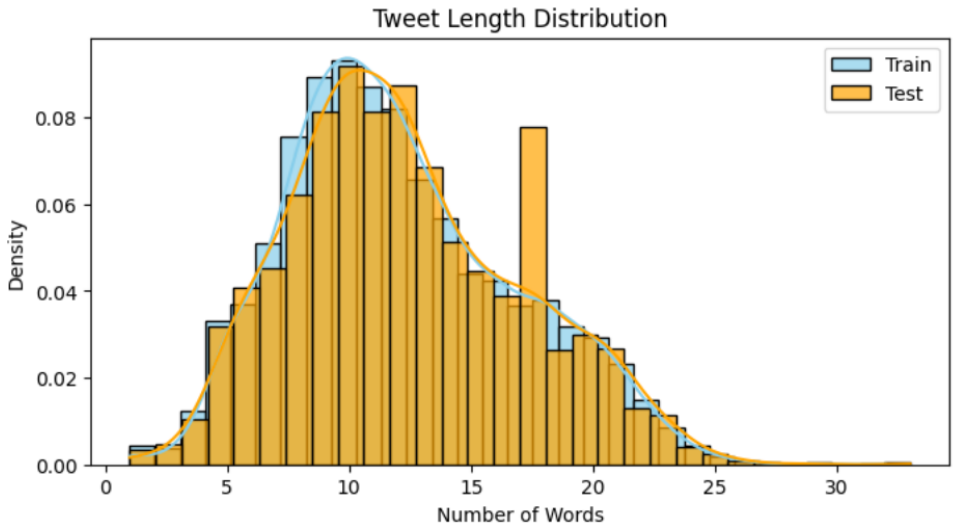


Figure 3: Distribution of tweet lengths (in words)

Text Preprocessing and Tokenization

Then, to prepare the data, I used the NLTK TreebankTokenizer, which splits text into tokens using a set of regular expressions useful to handle punctuation and linguistic patterns in English as I thought it might be well-suited for the short, informal style of tweets. I lowercased all text, removed URLs (as I expected them not to contribute to sentiment) and replaced ticker symbols and hashtags with placeholders to capture their presence while avoiding overfitting to specific cases. Emojis were standardized, punctuation was removed (except for exclamation/question marks, used as features), and words were lemmatized (to unify inflections and reduce feature sparsity). For

stopword removal, I retained financial negations and key sentiment words (“buy”, “sell”, etc.) to avoid discarding important context.

```
Original:  $BYND - JPMorgan reels in expectations on Beyond Meat https://t.co/bd0xbFGjkT
Processed: TICKER_SYMBOL jpmorgan reel expectation beyond meat
--------------------------------------------------------------------------------
Original:  $CCL $RCL - Nomura points to bookings weakness at Carnival and Royal Caribbean https://t.co/yGjpT2ReD3
Processed: TICKER_SYMBOL TICKER_SYMBOL nomura point booking weakness carnival royal caribbean
--------------------------------------------------------------------------------
Original:  $CX - Cemex cut at Credit Suisse, J.P. Morgan on weak building outlook https://t.co/KN1g4AWFIb
Processed: TICKER_SYMBOL cemex cut credit suisse morgan weak building outlook
--------------------------------------------------------------------------------
```

Figure 4: Examples of original tweets and their processed (cleaned) versions after text preprocessing.

After preprocessing, I computed vocabulary statistics and extracted the most frequent tickers as seen in Figure 5 and 6 and hashtags which confirmed the strong financial focus of the dataset.



Figure 5: Top 10 most frequent stock tickers and hashtags in the dataset.



Figure 6: Word cloud of the most common words after preprocessing

Feature Engineering

Subsequently, I used two main types of text representations. First, I extracted **TF-IDF features**, including up to 8,000 n-grams (unigrams, bigrams, and trigrams), to capture important terms and short phrases related to sentiment. Second, for the neural approach, I generated **FinBERT** embeddings—768-dimensional vectors created by a BERT-based model pre-trained on financial texts—so that each tweet is mapped to a dense, contextual representation. FinBERT is a variant of BERT (Bidirectional Encoder Representations from Transformers) specifically trained on financial documents and news, enabling it to better capture domain-specific semantics and sentiment signals in financial language. In both approaches, I further combined these text features with a set of **manual financial features**. These include counts of bullish and bearish sentiment words (with the word lists generated using large language models), a sentiment score (bullish minus bearish word count), and binary indicators for the presence of tickers, hashtags, dollar amounts, percentages, exclamation/question marks, and the capitalization ratio. The table in Figure 7 summarizes the distribution of these manual features in the dataset. I tested both model variants several times with and without the manual features and found that including them consistently led to slightly better results.

	Mittelwert	Std	Min	Median	Max
bullish_word_count	0.166	0.411	0.00	0.000	3.000
bearish_word_count	0.064	0.258	0.00	0.000	3.000
sentiment_score	0.047	0.224	-0.75	0.000	0.750
has_ticker	0.150	0.357	0.00	0.000	1.000
has_percentage	0.087	0.282	0.00	0.000	1.000
has_dollar_amount	0.111	0.314	0.00	0.000	1.000
has_hashtag	0.094	0.292	0.00	0.000	1.000
exclamation_count	0.017	0.258	0.00	0.000	14.000
question_count	0.058	0.246	0.00	0.000	6.000
caps_ratio	0.121	0.080	0.00	0.113	0.867

Figure 7: Summary statistics of engineered financial features

Model Training and Theory

Logistic Regression:
For the first model, I trained a logistic regression classifier on the combined TF-IDF features and manual financial features. Logistic regression models the probability of each class k via the softmax function over a linear combination of features:

$$P(y = k \mid x) = \frac{\exp(\mathbf{w}_k{}^T \mathbf{x} + b_k)}{\sum_{j=1}^{C} \exp(\mathbf{w}_j{}^T \mathbf{x} + b_j)}$$

where x is the feature vector, and C is the number of classes. I used balanced class weights to address the class imbalance in the dataset.

Neural Model (MLP):
For the second model, I trained a multi-layer perceptron (MLP) with three hidden layers of sizes 512, 256, and 64, using the concatenated FinBERT embeddings and manual financial features as input. The MLP consists of fully connected layers with ReLU activations and a softmax output layer for multiclass classification, allowing the network to model non-linear relationships between features.
I used early stopping to prevent overfitting and applied SMOTE oversampling to create a more balanced training set by generating synthetic samples for minority classes. The idea was that by learning non-linear transformations of the input, the MLP can potentially capture more complex and context-dependent patterns in the data compared to a linear model.

Evaluation Metrics

Since the dataset is imbalanced, I relied on multiple evaluation metrics. While **accuracy** (overall proportion of correct predictions) provides a basic measure, it can be misleading in imbalanced settings, as a model could achieve high accuracy simply by predicting the majority class. Therefore, I also used **precision** (correctness among positive predictions), **recall** (coverage of true instances), and **F1-score** (harmonic mean of precision and recall) to assess model performance for each class individually. To further account for class imbalance, I reported **macro averages** (treats all classes equally) and **weighted averages** (weights classes by their frequency). Finally, I used the **confusion matrix** (shows the breakdown of prediction errors for each class) to better understand where the model struggle.

Results

Both models performed well above random guessing, as reflected by high accuracy and F1-scores. The logistic regression model achieved an accuracy of 0.809, macro F1 of 0.753, and weighted F1 of 0.814. It performed best on Neutral tweets but showed notable confusion between Bearish and Neutral classes—likely a consequence of class imbalance, as also visible in the confusion matrix.

The neural model reached a similar overall accuracy (0.802), with a macro F1 of 0.725 and weighted F1 of 0.797. Precision for Bullish tweets improved but recall for Bearish was still lower despite SMOTE balancing, suggesting that minority classes remain challenging.

	precision	recall	f1-score	support
Bearish	0.600	0.712	0.651	347
Bullish	0.693	0.781	0.735	475
Neutral	0.912	0.839	0.874	1566
accuracy	0.809	0.809	0.809	1
macro avg	0.735	0.777	0.753	2388
weighted avg	0.823	0.809	0.814	2388

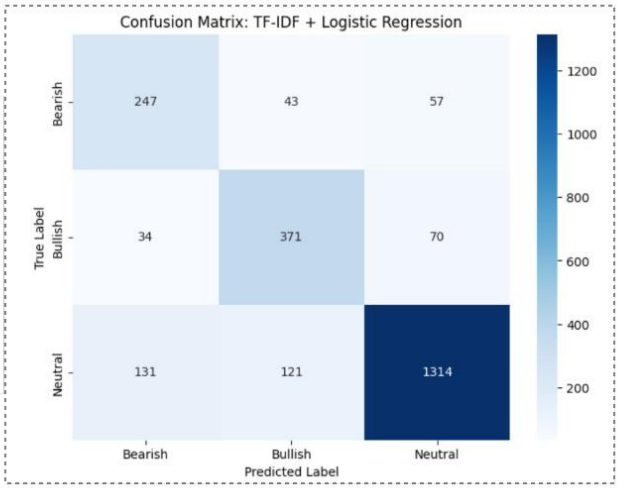


Figure 8: Results and confusion matrix for TF-IDF + logistic regression

	precision	recall	f1-score	support
Bearish	0.668	0.585	0.624	347
Bullish	0.719	0.642	0.679	475
Neutral	0.847	0.898	0.872	1566
accuracy	0.802	0.802	0.802	1
macro avg	0.745	0.708	0.725	2388
weighted avg	0.796	0.802	0.797	2388

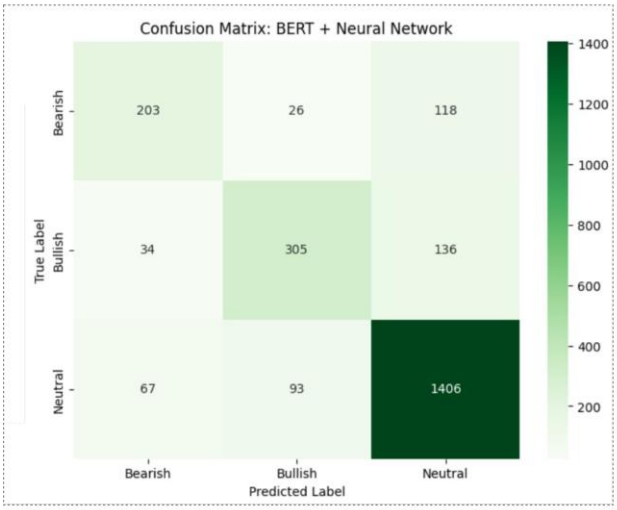


Figure 9: Results and confusion matrix for FinBERT + neural network

Comparing both models, logistic regression slightly outperformed the neural net across most metrics. I suspect this may be due to the nature of financial tweets, where explicit signals—like tickers, sentiment keywords, and formulaic expressions—are directly picked up by n-gram and manual features, which might give logistic regression a natural advantage. In contrast, the neural model's potential to leverage deeper context did not seem to offer a clear benefit in this short-text scenario. I believe this could be related to limited data or the simple and repetitive language typical of tweets.

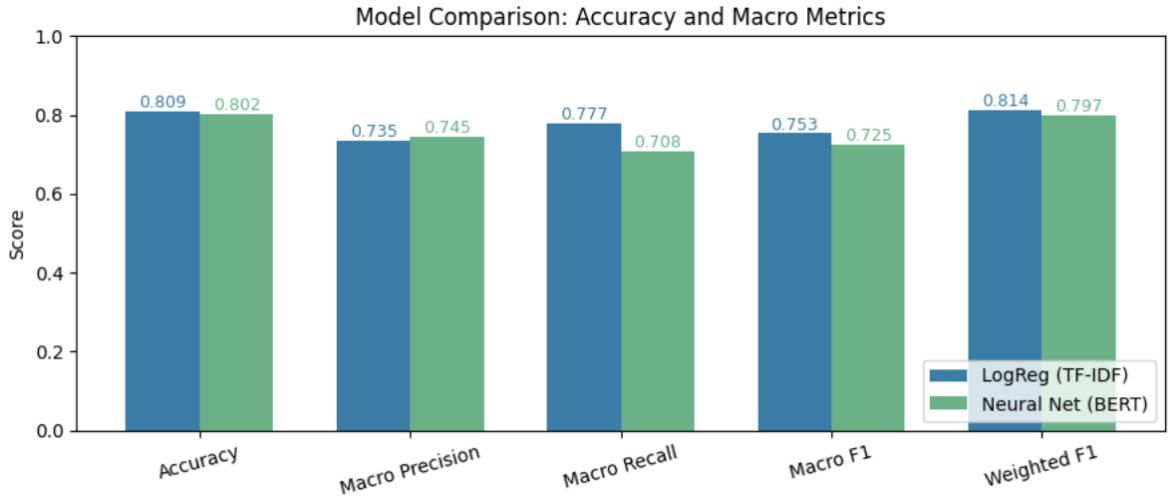


Figure 10: Side-by-side comparison of model performance metrics

Going forward, I see several possible ways to potentially improve performance. Collecting more data—especially for minority classes—could help adress the class imbalance issue. In addition, I would consider experimenting with more diverse feature engineering strategies and exploring alternative or hybrid model architectures (e.g., ensembles that combine classical and neural approaches). Integrating advanced

sampling techniques might also enhance results. However, extensions like this go beyond the scope of this assignment.

Conclusion

In this assignment, I worked on classifying financial tweets by sentiment. After exploring and preprocessing the dataset, I chose two main text representations: TF-IDF features and FinBERT embeddings, each combined with manual financial features like sentiment word counts and ticker indicators. This allowed the models to use both general text patterns and finance-specific signals. Logistic regression with TF-IDF and manual features performed slightly better than the neural network with FinBERT, maybe because clear cues like tickers and sentiment words matter a lot for short tweets. The main challenge was the strong class imbalance, which made minority classes harder to predict. Overall, the project showed that with the right features, even simple models can deliver promising results for financial sentiment analysis. There's still room to improve, especially with more data for rare classes or by trying advanced models, but this approach is a solid starting point for using social media dynamics e.g. in trading such as market reactions in real world use cases like the GameStop share price rise, Elon Musk's tweets or posted statements from politicians.